

CodePoster

Rapport Complet

Jeu de Programmation Multijoueur en Temps Réel

Statut	Version 1.0 – MVP
Date	28 mai 2026
Équipe	Aymane EDDAMANE & Taha HOUMMADI
Diffusion	Rapport enseignant

Partie I Présentation générale – à l'intention du grand public

Partie II Rapport technique d'ingénierie – analyse architecturale

Table des matières

I	Présentation générale	2
1	En deux phrases – qu’est-ce que CodePoster ?	2
2	L’inspiration – pourquoi ce jeu existe	2
2.1	Among Us – le point de départ	2
2.2	L’idée de CodePoster	2
2.3	La valeur éducative	2
3	Les rôles – coéquipier ou imposteur ?	3
4	Comment se déroule une partie	3
4.1	Étape 1 – Le Lobby	3
4.2	Étape 2 – La salle d’attente	4
4.3	Étape 3 – Le briefing de mission	4
4.4	Étape 4 – La phase de jeu	4
4.5	Étape 5 – Fin de partie	5
5	Les cinq défis de code	5
6	L’interface du jeu	5
7	La collecte de données – un observatoire unique de l’apprentissage	6
7.1	Pourquoi ces données sont précieuses	6
7.2	Ce que le jeu observe	6
7.3	De la donnée brute à l’insight – exemples concrets	6
7.4	Marchés potentiels	7
8	L’équipe et la vision	7
II	Rapport technique d’ingénierie	8
1	Résumé exécutif	8
2	Architecture globale	8
2.1	Vue d’ensemble	9
2.2	Principes clés	9
3	Backend – Analyse technique	9
3.1	Technologies	9
3.2	Structure des packages	9
3.3	Flux principal	10
3.4	Sandbox d’exécution	10
3.5	Bugs identifiés dans le code source	10
4	Frontend – Analyse technique	11
4.1	Technologies	11
4.2	Navigation	11
4.3	Collaboration temps réel	11
4.4	UI et expérience utilisateur	11

5	Modèle de données	12
5.1	Entités principales	12
5.2	Stratégie de persistance	12
6	Flux temps réel (STOMP)	12
6.1	Destinations client → serveur (/app)	12
6.2	Destinations serveur → client	12
6.3	Diagramme de séquence – Cycle complet d’une partie	13
7	Sécurité et risques	13
7.1	État actuel	14
7.2	Recommandations	14
8	Qualité logicielle et tests	14
9	Performance, scalabilité et observabilité	14
9.1	Performance	14
9.2	Observabilité	15
10	Nouvelles fonctionnalités proposées	15
10.1	Service de collecte de données (Analytics)	15
10.1.1	Stratégie d’identification – username comme clé primaire	15
10.1.2	Valeur de marché	15
10.1.3	Cartographie des données collectées	15
10.1.4	Sélection réaliste – périmètre MVP	16
10.1.5	Document analytics produit par partie	16
10.1.6	Architecture recommandée	17
10.2	Assistant IA intégré	17
10.2.1	Diagramme d’intégration	17
11	Améliorations et feuille de route	17
11.1	Court terme – Corrections prioritaires	17
11.2	Moyen terme	18
11.3	Long terme	18
12	Annexes	18
12.1	Technologies (résumé)	18
12.2	Glossaire	18

Partie I

Présentation générale

1 En deux phrases – qu'est-ce que CodePoster ?

CodePoster est un **jeu en ligne multijoueur** dans lequel quatre joueurs travaillent ensemble pour réparer du code informatique buggué, pendant qu'un **imposteur** caché parmi eux sabote leurs efforts.

CodePoster s'inspire du jeu social *Among Us*, en transposant ses mécaniques dans l'univers de la programmation : les exercices de code remplacent les tâches spatiales, et l'ambiance cyber-futuriste remplace le vaisseau spatial.

L'idée en une image

Quatre joueurs. Un éditeur de code par joueur. Cinq fichiers corrompus à réparer. Un imposteur secret parmi vous. Trente minutes pour sauver le réseau.

2 L'inspiration – pourquoi ce jeu existe

2.1 Among Us – le point de départ

En 2020, le jeu *Among Us* a captivé des millions de joueurs dans le monde. Son concept est simple : une équipe de coéquipiers accomplit des tâches sur un vaisseau spatial, tandis qu'un ou plusieurs imposteurs tentent de saboter l'équipe discrètement. La tension sociale est la vraie mécanique du jeu.

2.2 L'idée de CodePoster

CodePoster transpose cette mécanique dans le monde de la programmation. Les *tâches* deviennent des **exercices de débogage Python**. Le vaisseau spatial devient une **station de code collaborative**. L'imposteur ne sabote plus un vaisseau – il **corrompt le code** de ses coéquipiers à distance.

Jouer à un jeu de société tout en apprenant à programmer. Résoudre des problèmes informatiques réels sous pression sociale et temporelle. C'est à la fois un divertissement et un outil pédagogique.

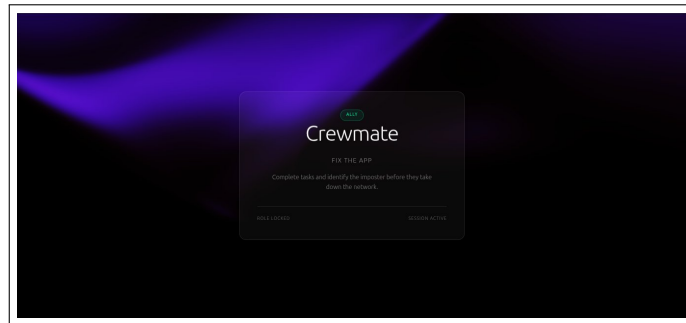
2.3 La valeur éducative

En jouant à **CodePoster**, on développe sans s'en rendre compte :

- **Résolution de problèmes** : identifier et corriger des erreurs dans du code réel.
- **Travail sous pression** : coder avec un chronomètre qui tourne et un imposteur à démasquer.
- **Esprit critique** : distinguer un allié d'un imposteur à partir de son comportement.
- **Collaboration** : partager un éditeur de code en temps réel avec ses coéquipiers.

3 Les rôles – coéquipier ou imposteur ?

Chaque joueur reçoit un rôle **secret** au début de la partie, révélé uniquement à lui. Personne d'autre ne sait qui est qui.



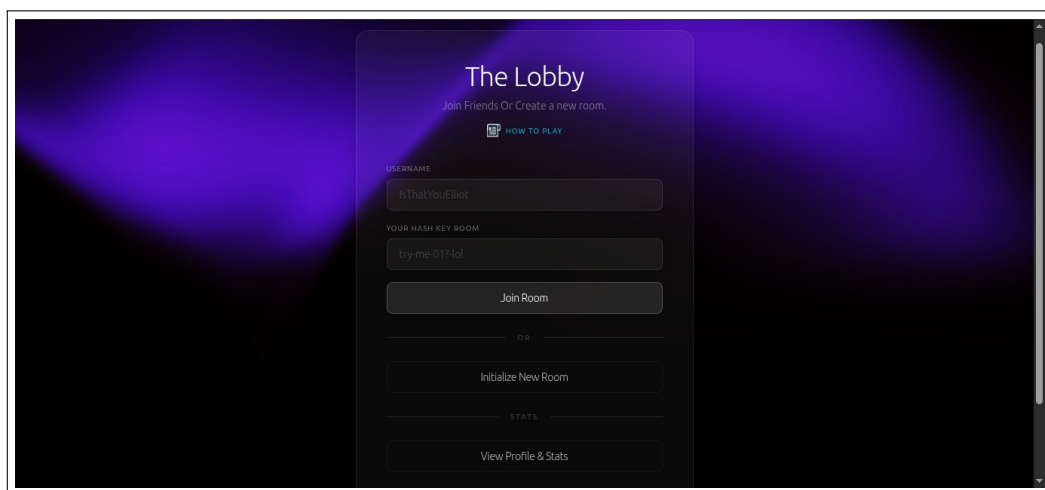
Écran de révélation du rôle – le joueur apprend en privé qu'il est Crewmate (Coéquipier). L'imposteur voit un écran similaire en rouge.

Son objectif : régler ses cinq exercices de code le plus vite possible **OU** démasquer l'imposteur par le vote. Il peut déclencher une réunion d'urgence s'il soupçonne un autre joueur.

Il dispose d'un pouvoir de **sabotage** : cibler un coéquipier et corrompre son fichier à distance. Il doit paraître normal, participer aux discussions, et convaincre les autres de voter contre un innocent. Son objectif : ne pas être éjecté.

4 Comment se déroule une partie

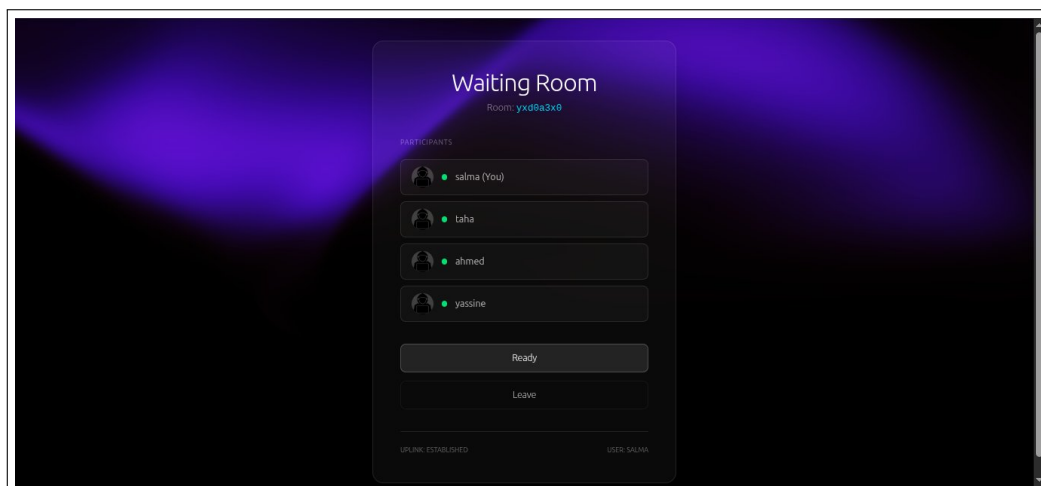
4.1 Étape 1 – Le Lobby



Le Lobby – le joueur saisit son pseudonyme et une clé de salle pour rejoindre ses amis, ou crée une nouvelle salle.

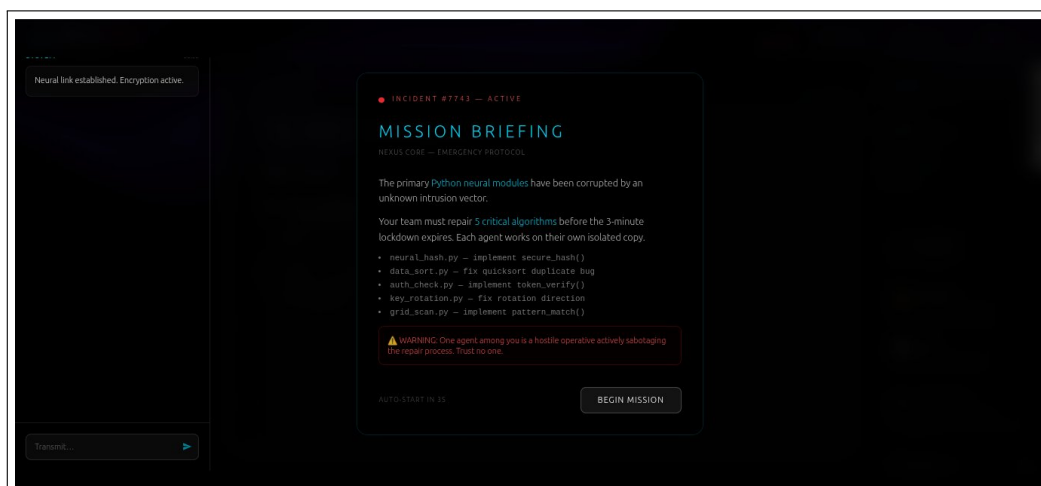
Le joueur choisit un pseudonyme et rejoint une salle existante via sa *clé de salle*, ou en crée une nouvelle. La salle accueille jusqu'à 4 joueurs.

4.2 Étape 2 – La salle d’attente



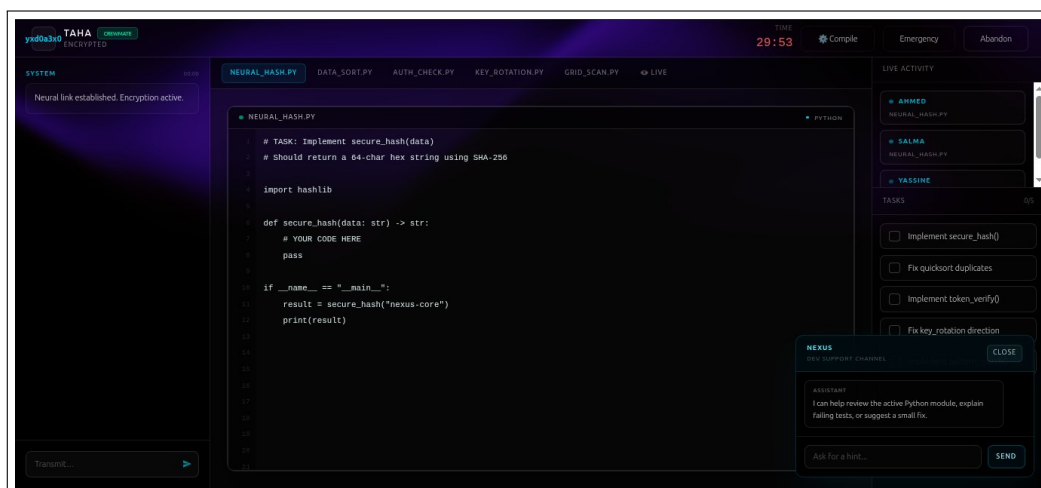
La Waiting Room – les quatre joueurs (salma, taha, ahmed, yassine) sont connectés. La partie se lance quand tous cliquent sur « Ready ».

4.3 Étape 3 – Le briefing de mission



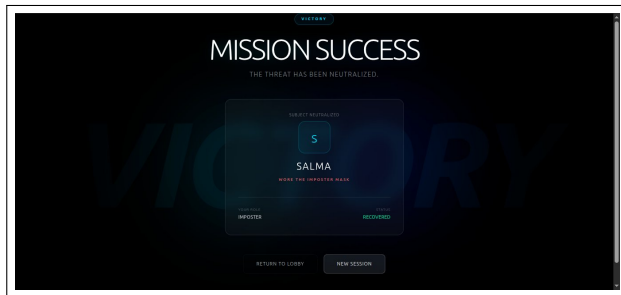
Le Mission Briefing – les cinq fichiers corrompus sont listés. L’avertissement en rouge rappelle discrètement qu’un imposteur se cache parmi l’équipe.

4.4 Étape 4 – La phase de jeu

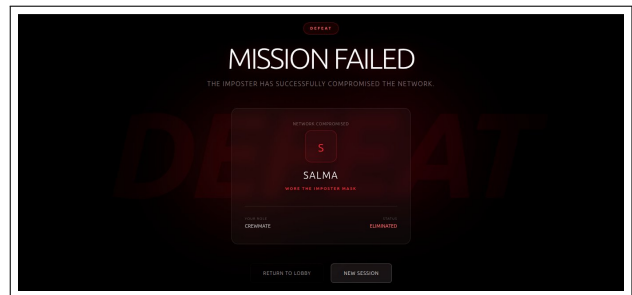


L'écran de jeu – éditeur de code au centre, onglets des 5 fichiers, panel Live Activity (activité en temps réel des coéquipiers), assistant IA ouvert en bas à droite.

4.5 Étape 5 – Fin de partie



VICTORY – L'imposteur (Salma) a été neutralisé.



DEFEAT – L'imposteur a compromis le réseau.

L'écran de fin révèle le rôle de chaque joueur avec une ambiance visuelle distincte (bleu pour la victoire, rouge pour la défaite).

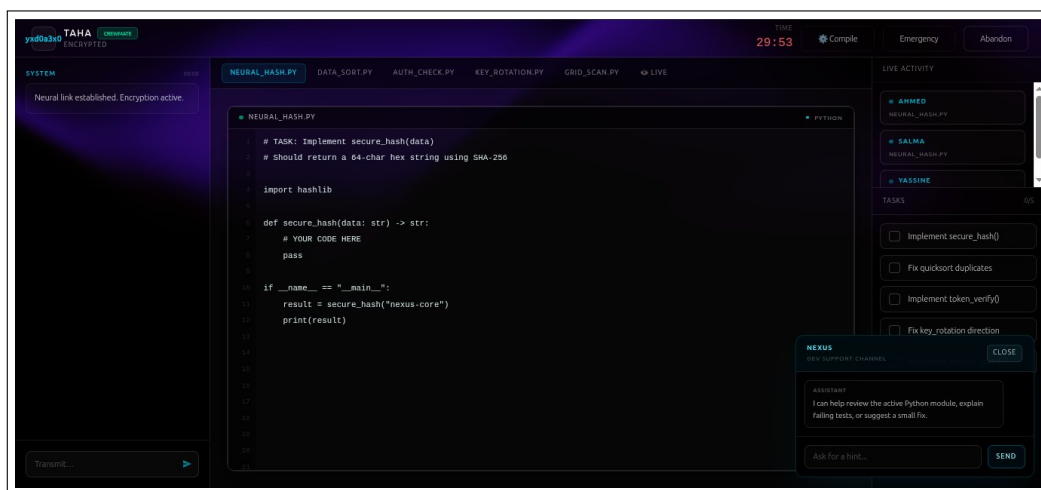
5 Les cinq défis de code

Fichier	Thème	Ce que le joueur doit faire
neural_hash.py	Hachage	Implémenter une fonction d'empreinte unique
data_sort.py	Tri	Corriger un algorithme de tri rapide (quicksort) buggué
auth_check.py	Vérification	Implémenter la validation d'un code d'accès
key_rotation.py	Rotation	Corriger la direction de rotation d'une clé
grid_scan.py	Exploration	Implémenter un parcours de grille (pattern match)

Pas besoin d'être expert

Les défis sont conçus pour des étudiants de première ou deuxième année en informatique. Il ne s'agit pas d'écrire du code de zéro, mais de **trouver et corriger des erreurs** dans du code partiellement écrit. Le jeu indique si la solution est correcte en quelques secondes.

6 L'interface du jeu



Vue complète de l'écran de jeu : panneau de chat (gauche), éditeur de code central avec les 5 onglets, panel droit avec l'activité en direct et la liste des tâches.

- **Les onglets** (en haut) : un par fichier Python + onglet LIVE pour observer un coéquipier.
- **L'éditeur** (centre) : le fichier Python à corriger avec syntaxe colorée.
- **Le chronomètre** (haut droite) : compte à rebours partagé par tous.
- **Live Activity** (droite) : sur quel fichier travaille chaque coéquipier en temps réel.
- **L'assistant IA** (panneau flottant) : demander un indice sans quitter l'éditeur.
- **Emergency** (bouton haut) : déclenche une réunion d'urgence pour voter.

7 La collecte de données – un observatoire unique de l'apprentissage

7.1 Pourquoi ces données sont précieuses

CodePoster n'est pas seulement un jeu. C'est un **observatoire naturel** du comportement humain face à la programmation. Le joueur est dans un état authentique : sous pression, en collaboration, prenant des décisions en temps réel.

Les plateformes classiques mesurent *si* un étudiant a réussi un exercice. **CodePoster** mesure *comment* il l'a résolu : dans quel ordre, avec quelle vitesse, après combien d'erreurs, avec quelle réaction au stress social. Ces données comportementales, collectées dans un contexte ludique et non simulé, offrent une fenêtre authentique sur les mécanismes d'apprentissage – une qualité introuvable dans tout autre outil d'évaluation existant.

7.2 Ce que le jeu observe

Résultats de partie	Comportement code	Comportement social
Qui a gagné / perdu	Temps par exercice	Votes corrects
Durée de la partie	Exercices réussis	Réunions initiées
Nombre de sabotages	Erreurs rencontrées	Ordre de complétion

7.3 De la donnée brute à l'insight – exemples concrets

Donnée observée	Insight produit
80% des joueurs passent >8 min sur grid_scan.py	Ce concept est trop difficile pour le niveau cible
Alice change de fichier 12 fois en 10 min	Style dispersé – conseil : rester focalisé
Bob réussit 4 exercices sur 5 en 15 min	Niveau avancé, il peut être orienté vers des défis supplémentaires
Taux de compilation réussie : 40% sur auth_check	L'énoncé est trop ambigu, à reformuler

7.4 Marchés potentiels

Marché	Valeur apportée
EdTech	Identifier les concepts Python difficiles et adapter le programme.
RH / Recrutement	Évaluer compétences techniques et soft skills en situation réelle.
Recherche	Données de comportement de codage collaboratif sous contrainte sociale.
Gaming	Segmenter les joueurs par profil pour adapter la difficulté.

8 L'équipe et la vision

CodePoster a été développé par **Aymane EDDAMANE** et **Taha HOUMMADI**, étudiants ingénieurs à l'**ENSAM Meknès**. Le prototype est pleinement fonctionnel : connexion multijoueur, rôles secrets, éditeur collaboratif, votes, sabotage et assistant IA.

Vision à moyen terme : comptes joueurs permanents, nouvelles catégories d'exercices, mode tournoi, dashboard enseignant, recommandations personnalisées par l'IA basées sur les données accumulées.

CodePoster transforme la frustration du débogage en une mécanique sociale engageante, tout en collectant des données précieuses sur l'apprentissage de la programmation – des données qu'aucun autre outil ne peut produire.

Partie II

Rapport technique d'ingénierie

Note de lecture

Le contexte fonctionnel, les règles du jeu et la valeur pédagogique de **CodePoster** sont présentés en détail dans la Partie I de ce document. Cette partie se concentre exclusivement sur l'analyse technique et architecturale du système.

1 Résumé exécutif

CodePoster s'appuie sur :

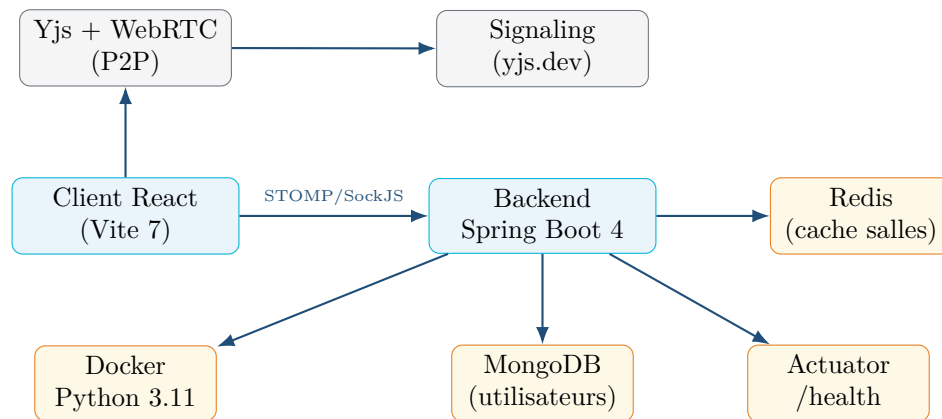
- un backend Java Spring Boot (WebSocket/STOMP + REST) ;
- un frontend React/Vite avec UI riche et animations ;
- une collaboration P2P via Yjs + WebRTC ;
- MongoDB pour la persistance des joueurs ;
- Redis pour le cache des listes de joueurs par salle ;
- un sandbox Docker pour exécuter du code Python.

- Collaboration P2P native via Yjs CRDT + WebRTC – aucune charge serveur pour la synchronisation du code.
- Sandbox Docker isolé avec limites CPU/mémoire – bonne maîtrise des risques d'exécution.
- Stack événementielle cohérente : STOMP sur WebSocket pour tous les événements de jeu.
- Technologies récentes : Java 21, Spring Boot 4.0.3, React 19, Vite 7, Tailwind 4.

- Credentials MongoDB codés en dur dans `application.properties`.
- Test runner Python codé en dur (`pattern_match`) – pas encore généralisé aux 5 tâches.
- `readyNumber()` globale – bug potentiel en multi-salle.
- Pas d'authentification – identification par `username` uniquement (choix MVP).
- Serveur de signaling Yjs public (`yjs.dev`) – dépendance externe non contrôlée.

2 Architecture globale

2.1 Vue d'ensemble



2.2 Principes clés

- **Temps réel** : STOMP au-dessus de WebSocket pour les événements de jeu.
- **Collaboration P2P** : Yjs CRDT pour la synchronisation de texte sans charge serveur.
- **Persistant + cache** : MongoDB (utilisateurs) + Redis (listes par salle).
- **Sandbox d'exécution** : Docker exécute le code Python en isolation totale.

3 Backend – Analyse technique

3.1 Technologies

Technologie	Version	Rôle
Java	21 (LTS)	Langage principal
Spring Boot	4.0.3	Framework applicatif
Spring WebSocket STOMP	+ –	Messagerie temps réel
Spring Data MongoDB	–	Persistance utilisateurs
Spring Data Redis	–	Cache listes de joueurs
Spring Boot Actuator	–	Health check /actuator/health
Lombok	Stable	Réduction boilerplate Java
Docker (python :3.11-slim)	Runtime	Sandbox d'exécution isolé

3.2 Structure des packages

```

com.example.websocketchat
|-- config/
| |-- WebSocketConfig.java (endpoint /ws, prefixes /app /topic /queue)
| |-- RedisConfig.java
| |-- CustomHandshakeInterceptor (injection sessionId)
| '-- WebSocketEventListener (deconnexion + suppression utilisateur)
-- controller/
| |-- RoomController.java (join, vote, sabotage, task, game-over)
| |-- ChatController.java (messages salle d'attente)
| |-- CompileController.java (REST POST /api/compile)
| '-- UserController.java
-- service/
| |-- RoomService.java (roles, votes, taches, fin de partie)
| |-- CompilerService.java (execution Python via Docker)
  
```

```
| |-- MessagingService.java (sendToTopic / sendToUser)
| '-- userService.java (CRUD + cache Redis)
|-- model/
| |-- User.java (@Document MongoDB)
| |-- ChatMessage.java (types: CHAT, VOTE, SABOTAGE, ...)
| |-- Task.java / TestCase.java
| |-- Submission.java
| '-- Sabotage.java
'-- repository/
    |-- UserRepository.java
    '-- TaskRepository.java
```

3.3 Flux principal

1. Connexion STOMP avec identifiant utilisateur.
2. Ajout du joueur en salle (attente ou jeu).
3. Attribution des rôles et début de partie.
4. Gestion des événements (chat, sabotage, vote).
5. Fin de partie (vote ou timer).
6. Déconnexion : suppression automatique de l'utilisateur en MongoDB.

3.4 Sandbox d'exécution

```
// CompilerService.java -- Parametres Docker
"docker", "run",
    "--rm", // suppression auto
    "--network", "none", // reseau desactive
    "--memory", "100m", // limite RAM
    "--cpus", "0.5", // limite CPU
    "-v", tempDir + ":/app",
    "python:3.11-slim",
    "python", "test_runner.py"
// Timeout : 3 secondes (process.waitFor)
```

Le `test_runner.py` est codé en dur pour tester `pattern_match()` uniquement. Il doit être généralisé pour charger dynamiquement les cas de test des 5 tâches depuis `TaskRepository`.

3.5 Bugs identifiés dans le code source

La méthode `userService.readyNumber()` compte **tous** les utilisateurs *ready* en base, sans filtre par salle. En contexte multi-salle, cela pourrait déclencher prématurément une partie.

Correction : ajouter un filtre `roomId` dans la requête MongoDB.

La méthode `findByRoom()` est annotée `@CachePut`, ce qui signifie qu'elle écrit toujours en cache mais ne lit pas depuis le cache. Changer en `@Cacheable` pour éviter des requêtes Mongo inutiles à chaque appel.

4 Frontend – Analyse technique

4.1 Technologies

Technologie	Version	Rôle
React	19.2.0	Librairie UI
React Router	7.13.0	Navigation SPA
Vite	7.2.4	Build tool
Tailwind CSS	4.1.18	Styling utilitaire
GSAP + @gsap/react	3.14.2	Animations avancées
Three.js + @react-three/fiber	0.182.0 / 9.5.0	Rendus 3D / shaders
Lottie-react	2.4.1	Animations JSON
SockJS-client + stompjs	1.6.1 / 2.3.3	Client WebSocket STOMP
Yjs	13.6.30	CRDT collaboration temps réel
y-webrtc	10.3.0	Transport WebRTC pour Yjs
lucide-react	0.563.0	Icônes

4.2 Navigation

- `/login` – LoginPage (saisie du username, côté frontend uniquement – pas d’endpoint backend).
- `/` – LobbyPage (créer / rejoindre salle).
- `/create` – CreateRoomPage.
- `/waiting/:roomId` – WaitingRoomPage (prêt, affichage joueurs).
- `/room/:roomId` – RoomPage (jeu complet).
- `/role` – RolePage (révélation du rôle secret).
- `/reveal` – RevealPage (fin de partie).
- `/profile` – ProfilePage (stats – statique pour l’instant).

La page de connexion existe côté frontend et permet au joueur de saisir son **username**. Il n’existe ni `POST /api/login`, ni vérification d’identité côté serveur. Le **username** est stocké localement (état React) et transmis directement dans les messages STOMP. Un endpoint d’authentification sera introduit lors de l’implémentation du système JWT.

4.3 Collaboration temps réel

- Yjs crée un document partagé (**Y.Doc**) par salle.
- Chaque fichier est un **Y.Text** synchronisé P2P automatiquement.
- Awareness partage l’activité (fichier actif, typing).
- Mode *Live* permet de regarder un joueur en lecture seule.

4.4 UI et expérience utilisateur

- Design cyber / futuriste cohérent sur l’ensemble des pages.

- Effets glitch, shader Three.js et neural network animé sur la page d'accueil.
- Animations Lottie contextuelles : emergency, compile, countdown, recording.
- Panels modulaires : activité en direct, sabotage, vote, compilation.
- Sons d'ambiance (`lightning.mp3`) pour les événements critiques.

5 Modèle de données

5.1 Entités principales

- **User** (@Document MongoDB) : `username`, `waitingroomId`, `ready`, `Imposter`, `roomOwner`, `sessionId`.
- **ChatMessage** : `type` (CHAT, JOIN, LEAVE, READY, START_GAME, TASK_COMPLETE, VOTE, VOTE_RESULT, SABOTAGE, GAME_OVER, EMERGENCY_MEETING...), `content`, `sender`, `target`, `sabotage`.
- **Task** / **TestCase** : modèles existants, non encore chargés dynamiquement.
- **Submission** : fichiers envoyés pour compilation.

5.2 Stratégie de persistance

Données	Stockage	Durée de vie
Utilisateurs	MongoDB Atlas	Jusqu'à déconnexion (<code>WebSocketEventListener</code>)
Listes de joueurs	Redis Cache	Jusqu'à invalidation
Votes en cours	ConcurrentHashMap	Durée du vote
Progression tâches	ConcurrentHashMap	Durée de la partie
Code collaboratif	Yjs P2P (mémoire)	Session WebRTC
Tâches de débogage	Hardcodé frontend	N/A (à migrer)

6 Flux temps réel (STOMP)

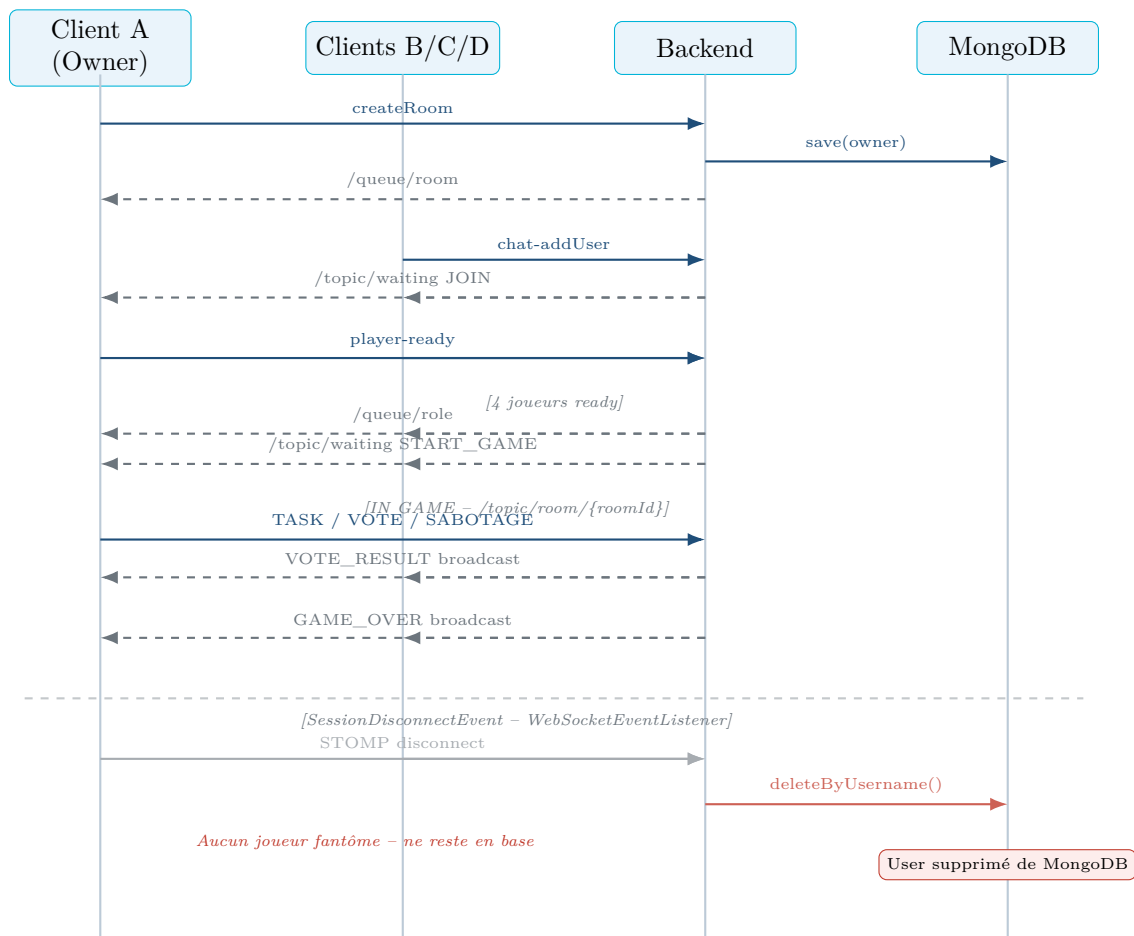
6.1 Destinations client → serveur (/app)

Destination	Rôle
<code>/app/createRoom/{roomId}</code>	Création de salle
<code>/app/chat-addUser/{roomId}</code>	Rejoindre salle d'attente
<code>/app/player-ready/{roomId}</code>	Indiquer prêt
<code>/app/room/{roomId}/join</code>	Rejoindre salle de jeu
<code>/app/room/{roomId}/chat</code>	Chat en partie
<code>/app/room/{roomId}/task</code>	Tâche terminée
<code>/app/room/{roomId}/vote</code>	Vote d'éjection
<code>/app/room/{roomId}/sabotage</code>	Sabotage
<code>/app/room/{roomId}/emergency</code>	Réunion d'urgence
<code>/app/room/{roomId}/compile-vote-start</code>	Initier un vote de compilation
<code>/app/room/{roomId}/compile-vote</code>	Voter pour la compilation
<code>/app/room/{roomId}/game-over</code>	Fin de partie (timer)

6.2 Destinations serveur → client

Destination	Type	Description
/topic/waiting/{roomId}	Public	JOIN, READY, START_GAME
/topic/room/{roomId}	Public	Tous les événements de jeu
/queue/role	Privé	Rôle secret assigné
/queue/room	Privé	Confirmation création salle
/queue/errors	Privé	Erreurs (salle pleine, username pris)
/queue/room/{roomId}/sabotage	Privé	Sabotage ciblé

6.3 Diagramme de séquence – Cycle complet d'une partie



Gestion de la déconnexion – `WebSocketEventListener`

Lors de la déconnexion d'un joueur, Spring intercepte `SessionDisconnectEvent`, récupère le `username` et appelle `userService.deleteByUsername()` qui supprime l'entrée en MongoDB. Aucun joueur fantôme ne reste persisté entre deux parties.

7 Sécurité et risques

7.1 État actuel

Point	État	Risque
Credentials MongoDB	URI en clair dans <code>application.properties</code>	CRITIQUE
Authentification	Aucune – username libre (choix MVP)	CONNU
Autorisation STOMP	Aucune	ÉLEVÉ
CORS	Limité à <code>localhost:5173</code>	Faible (dev)
Exécution Python	Docker isolé, <code>-network none</code>	Maîtrisé
Signaling Yjs	Serveur public <code>yjs.dev</code>	MOYEN
Fichiers temp	Non supprimés après exécution	MOYEN

7.2 Recommandations

Déplacer l'URI MongoDB dans des variables d'environnement :

```
spring.mongodb.uri=${MONGODB_URI}
```

Ne jamais commiter des credentials en clair dans un dépôt Git.

- Valider côté backend que le `sender` correspond bien au `sessionId` WebSocket.
- Limiter la création de salles : un seul `createRoom` par `sessionId` actif.

8 Qualité logicielle et tests

Catégorie	Couverture	Commentaire
Tests backend	Minimale	Uniquement chargement de contexte Spring
Tests de services	Absents	RoomService, CompilerService non testés
Tests STOMP	Absents	Aucun test de flux temps réel
Tests frontend	Absents	Aucun test unitaire ou E2E
Linting	ESLint	<code>eslint.config.js</code> présent
CI/CD	Absent	Aucun pipeline d'intégration continue

Recommandations : tests unitaires JUnit 5 + Mockito (`handleVote()`, `handleTaskComplete()`, `CompilerService`), tests d'intégration STOMP, tests E2E Playwright.

9 Performance, scalabilité et observabilité

9.1 Performance

- STOMP simple broker adapté à un petit nombre de salles simultanées.
- Yjs en P2P limite la charge serveur mais dépend du réseau client.
- Compilation Docker impacte CPU/RAM (limites déjà en place).

Pistes : broker externe (RabbitMQ/Redis Pub/Sub), file de compilations, TTL Redis pour les salles terminées, index MongoDB sur `waitingroomId`.

9.2 Observabilité

Actuator expose `/actuator/health`. Les logs sont des `System.out.println` non structurés. Plan d'amélioration : SLF4J + Logback (haute priorité), Prometheus + Grafana (moyenne), OpenTelemetry (basse), Sentry frontend (moyenne).

10 Nouvelles fonctionnalités proposées

10.1 Service de collecte de données (Analytics)

10.1.1 Stratégie d'identification – username comme clé primaire

Dans la version MVP, les joueurs sont identifiés uniquement par leur `username`. Le service analytics adopte cette contrainte comme clé primaire de toutes les métriques.

Choix MVP – username comme identifiant

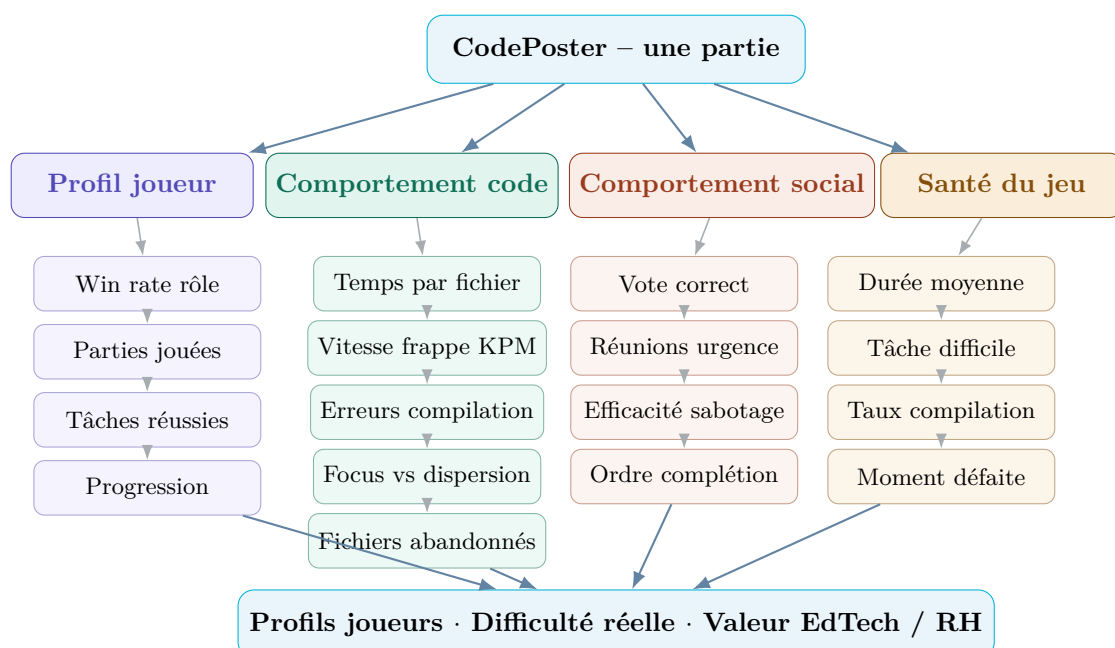
- Données indexées sur le couple (`username`, `roomId`).
- Un même username sur plusieurs parties = même joueur (dans la limite du MVP).
- **Limite connue** : deux joueurs différents avec le même username verront leurs statistiques fusionnées. Ce risque est acceptable pour le MVP et sera résolu par l'authentification JWT.

10.1.2 Valeur de marché

Marchés potentiels

- **EdTech** : identifier quels concepts Python posent le plus de difficulté.
- **RH / Recrutement** : évaluation comportementale sous contrainte sociale.
- **Gaming analytics** : modélisation et segmentation du comportement joueur.
- **Recherche académique** : patterns de collaboration en codage distribué.

10.1.3 Cartographie des données collectées



10.1.4 Sélection réaliste – périmètre MVP

Données disponibles immédiatement (zéro code) :

Donnée	Source STOMP	Valeur stockée
Résultat de la partie	GAME_OVER	outcome : WIN / LOSS / TIMEOUT
Rôle du joueur	/queue/role	crewmate ou imposter
Vote correct	VOTE_RESULT	ejectedIsImposter booléen
Durée	PLAYER_JOIN GAME_OVER	→ delta en secondes
Sabotages	SABOTAGE event	compteur par partie
Réunions urgence	EMERGENCY_MEETING	compteur par partie
Tâches complétées	TASK_COMPLETE	liste des fileKey réussis

Données faisables MVP (hook frontend + une ligne backend) :

Donnée	Implémentation	Effort
Temps par fichier Python	Timer onFileChange()	15 min
Nombre de frappes	Compteur onKeystroke()	10 min
Changements de fichier	Compteur onFileChange()	5 min
Résultat de compilation	POST /api/analytics/compile	20 min
Parties par username	Agrégation MongoDB	10 min

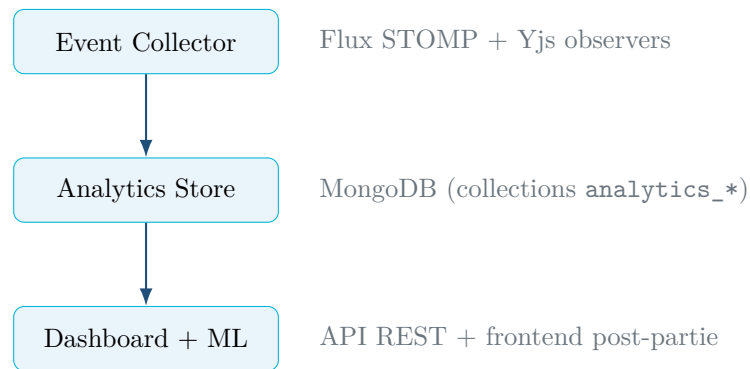
Données exclues du MVP :

Donnée exclue	Raison
Vitesse frappe KPM précise	Sans auth, username non lié à une session stable
Yjs Awareness détaillé	Sérialiser les données P2P demande un signaling custom
Efficacité du sabotage	Corrélation trop fragile sans timestamps précis
Contenu du code écrit	Poids de stockage et confidentialité
Stats multi-parties	Nécessite un identifiant stable – JWT d'abord

10.1.5 Document analytics produit par partie

```
{
  "roomId": "room-42", "startedAt": "2026-05-29T19:00:00Z",
  "endedAt": "2026-05-29T19:12:34Z", "durationSeconds": 754,
  "outcome": "CREWMATES_WIN", "totalSabotages": 2,
  "totalEmergencyMeetings": 1,
  "players": [{
    "username": "alice", "role": "crewmate", "won": true,
    "votedCorrectly": true,
    "tasksCompleted": ["neural_hash", "data_sort"],
    "timePerFile": {"neural_hash": 120, "data_sort": 95, "auth_check": 0},
    "keystrokesPerFile": {"neural_hash": 340, "data_sort": 210},
    "fileSwitches": 4,
    "compilationResults": {"neural_hash": "ACCEPTED", "data_sort": "WRONG_ANSWER"}
  }]
}
```

10.1.6 Architecture recommandée



Tout le pipeline analytics est asynchrone (@Async) et enveloppé dans un `try/catch` global. L'intercepteur STOMP retourne **toujours** le message intact, quelle que soit l'issue du traitement analytics.

10.2 Assistant IA intégré

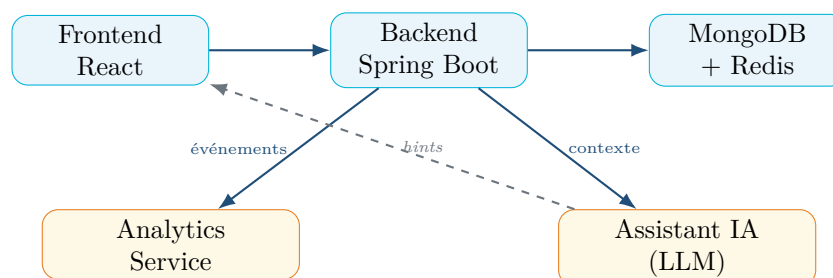
Mode	Déclencheur	Description
Hint engine	Bouton joueur	Indice contextuel sans donner la réponse
Analyse post-partie	Fin de partie	Rapport IA personnalisé par joueur
Bot imposteur	Mode solo	Jouer contre une IA

```

POST /api/hint
{ "username": "alice", "taskKey": "auth_check",
  "currentCode": "def auth_check(token):\n ...",
  "errorOutput": "AssertionError: line 5",
  "timeRemaining": 120, "hintsUsed": 1 }
  
```

Stack : Claude API (Anthropic) ou OpenAI GPT-4o. Endpoint `POST /api/hint` côté Spring Boot. Rate limiting : 3 indices max par joueur par partie.

10.2.1 Diagramme d'intégration



11 Améliorations et feuille de route

11.1 Court terme – Corrections prioritaires

- Sécuriser les credentials MongoDB via variables d'environnement.
- Mettre en service le pipeline de collecte de données analytics.
- Corriger `readyNumber()` pour filtrer par salle.

- Nettoyer les fichiers temporaires Docker (bloc `finally`).
- Changer `@CachePut` en `@Cacheable` sur `findByRoom()`.
- Brancher le `CompilerService` aux 5 tâches réelles du jeu.

11.2 Moyen terme

- Persister les tâches en base (`TaskRepository`).
- Implémenter l'authentification JWT.
- Tests unitaires backend (JUnit 5) + pipeline CI (GitHub Actions).
- Persister les statistiques joueurs (profil, historique).

11.3 Long terme

- Migration vers un broker externe (RabbitMQ / Redis Pub/Sub).
- Serveur de signaling Yjs auto-hébergé + tests E2E Playwright.
- Déploiement containerisé (Docker Compose / Kubernetes).
- Dashboard analytics Grafana + pipeline ML comportemental.
- Intégration LLM complète (hint engine + analyse post-partie).

12 Annexes

12.1 Technologies (résumé)

Domaine	Technologies
Backend	Java 21, Spring Boot 4.0.3, Spring WebSocket/STOMP, Spring Data MongoDB, Spring Data Redis, Lombok, Jackson, Spring Actuator
Frontend	React 19, React Router 7, Vite 7, Tailwind CSS 4, GSAP 3, Three.js, @react-three/fiber, Lottie-react, lucide-react
Temps réel	SockJS, stompjs, Yjs (CRDT), y-webrtc (WebRTC)
Sandbox	Docker, python :3.11-slim
Persistence	MongoDB Atlas (users), Redis (cache salles)
Build/Dev	Maven (backend), Vite + npm (frontend), ESLint

12.2 Glossaire

- **STOMP** : Simple Text Oriented Messaging Protocol – protocole de messagerie au-dessus de WebSocket.
- **CRDT** : Conflict-free Replicated Data Type – structure pour convergence sans conflit.
- **Yjs** : Implémentation JavaScript de CRDT pour la collaboration temps réel.
- **y-webrtc** : Provider Yjs utilisant WebRTC pour la communication P2P.
- **SockJS** : Librairie JavaScript offrant un fallback WebSocket sur navigateurs anciens.
- **Sandbox** : Environnement d'exécution isolé (Docker) pour limiter les risques.

- **SimpleBroker** : Broker STOMP en mémoire fourni par Spring – développement uniquement.
- **Crewmate** : Rôle coopératif – réparer les 5 fichiers ou identifier l’imposteur.
- **Imposteur** : Rôle adversarial – saboter sans être éjecté.
- **Awareness** : Fonctionnalité Yjs partageant l’état des utilisateurs connectés.
- **KPM** : Keystrokes Per Minute – métrique de vitesse de frappe.
- **TTL** : Time To Live – durée de vie d’une entrée en cache ou en base.
- **LLM** : Large Language Model – modèle de langage pour l’assistant IA.
- **SessionDisconnectEvent** : Événement Spring déclenché à la déconnexion STOMP.